

DUY TAN UNIVERSITY, DA NANG

DS 423 SA — Machine Learning with Large Datasets

Anime Recommendation System

Group 14

Individual contribution

Member	Primary contribution	Share
Mai Nam Quốc 29211145181	– Dataset loading, data cleaning and preprocessing, EDA charts, data preparation, APA reference compilation, and report review.	50%
Trương Nhật Trường 29211164915	– SVD model selection and implementation, evaluation, Top-10 export, presentation preparation, report integration, and conclusion drafting.	50%

July 2026

Abstract

Anime catalogues are large enough that manually finding titles aligned with a viewer’s preferences is difficult. This project builds a collaborative-filtering recommender that estimates a user’s preference for unseen anime and returns a Top-10 list. The Anime Recommendations Database provides 7,813,737 user–anime interactions and title metadata. Data preparation removed 1,476,496 watched-but-unrated records encoded as -1 , merged seven repeated user–anime pairs, and retained 6,337,234 valid explicit ratings on the original 1–10 scale. The resulting matrix is sparse (0.9172%), with positively skewed ratings and highly unequal user histories.

For a reproducible resource-aware experiment, the modeling notebook randomly sampled one million cleaned ratings, applied a minimum-five-interaction k-core filter, and trained a biased SVD model on an 80/20 outer holdout split. The model obtained RMSE 1.2632, MAE 0.9625, Precision@10 0.6386, and Recall@10 0.8255 under held-out observed-item ranking. These results describe the sampled and filtered evaluation population, rather than the whole catalogue. The system cannot fully address new users or new anime, and observed-item ranking is not equivalent to full-catalog ranking.

1 Project approach and team workflow

The group treated the recommender as a sequence of evidence-based decisions rather than as a single SVD training task. The initial question was whether the raw interaction table contained a valid preference signal; only after that question was answered did the group choose a model and an evaluation protocol. Table 1 records the reasoning chain, the ownership of each stage, and the artifact handed to the next stage. It therefore distinguishes the audited cleaned dataset from the smaller, model-specific dataset and distinguishes evaluation from the final recommendation-serving step.

Table 1: Team workflow: decision sequence, rationale, and handoff artifacts.

Stage	Question and decision	Reasoning and output	Lead / review
Define the task	What should be predicted and ranked? The group used explicit 1–10 user ratings to estimate preferences for unseen anime.	Missing interactions were treated as unknown rather than zero or negative ratings. This fixed the target, candidate meaning, and Top-10 objective.	Both define; Truong integrates
Audit and prepare data	Which rows are valid learning signals? Remove the watched-but-unrated sentinel –1, validate the 1–10 scale, and merge repeated user–anime pairs by their mean.	The resulting <code>ratings_clean.csv</code> preserves only explicit preferences and gives Notebook 02 a reproducible, auditable input.	Quốc leads; Truong checks boundary
Examine data reality	What constraints should influence modelling? Measure sparsity, rating skew, popularity concentration, and unequal user histories.	EDA showed a sparse, positive, long-tail matrix; it motivated collaborative filtering, explicit coverage reporting, and careful interpretation rather than an assumption that all users have equal evidence.	Quốc leads; both review
Model and test	How can the system be trained and assessed without leakage? Use a fixed-seed sample and k-core filter, then an outer 80/20 holdout and biased SVD fitted on training data only.	Notebook 02 records configuration, data scope, RMSE/MAE, and held-out observed-item Precision@10/Recall@10.	Truong leads; Quốc verifies inputs/results
Serve and integrate	How are recommendations made useful without changing the test result? Refit the fixed SVD on all model-scope interactions and rank only unseen candidates.	The export, report, figures, references, and presentation were cross-checked so that claims, metrics, and contribution shares remain consistent.	Truong leads serving/report integration; both review

1.1 Data and analysis approach: Quốc

The data-and-analysis work began by separating a data-quality question from a modelling question. Before deciding how to train a recommender, Quốc established what a valid user-preference observation meant and whether the raw files could support that meaning. The process was deliberately ordered: inspect first, define each cleaning rule from the meaning of the data, validate the resulting table, then use EDA to identify constraints that the later model must disclose. This avoided treating routine data transformations as assumptions hidden inside model code. Table 2 gives the detailed decision record.

Table 2: Detailed decision record for the data and EDA work led by Quốc.

Step	Evidence examined	Decision logic and action	Output / handoff
1. Establish the raw-data boundary	The two source tables and their required fields: user ID, anime ID, rating, and display metadata.	Read IDs without assuming they are numeric features; keep ratings observable until parsing so a malformed value cannot silently become a valid score. Verify schema, completeness, parseability, candidate-key duplicates, and metadata coverage before altering rows.	A before-cleaning quality snapshot for <code>rating.csv</code> and <code>anime.csv</code> .
2. Define a valid rating signal	The raw rating range and the 1,476,496 rows encoded as <code>-1</code> .	Interpret <code>-1</code> from the dataset semantics as “watched but not explicitly scored.” It gives no numerical preference direction, so retaining it would teach SVD that an unknown preference is worse than rating 1. Exclude it from both training and evaluation instead of converting it to zero or one.	An auditable removal count and a retained explicit 1–10 signal.
3. Resolve interaction ambiguity	Repeated (<code>user_id</code> , <code>anime_id</code>) keys after valid ratings were isolated.	A matrix factorisation model needs one target value per user–anime pair. Average the seven duplicate pairs rather than arbitrarily keeping the first or last row; accept fractional averages only because they remain inside the valid 1–10 range.	<code>ratings_clean.csv</code> with unique user–anime keys and scale assertions.
4. Pre-serve the metadata boundary	Metadata IDs, missing display fields, and one rated anime without matching metadata.	Keep metadata for title/genre display only. Do not impute missing labels or use public <code>anime_average_rating</code> as a training label, because doing either would blur catalogue context with a user’s explicit score. Retain valid ratings even when a display fallback will be needed later.	<code>anime_clean.csv</code> , an unmatched-metadata audit, and a clear serving fallback requirement.
5. Test the cleaned result	Counts, rating bounds, duplicate keys, and the post-cleaning user–anime matrix.	Use assertions to verify that no <code>-1</code> , missing required value, out-of-range rating, or duplicate interaction remains. This confirms that the cleaned table is a reproducible baseline rather than an informal spreadsheet edit.	6,337,234 explicit interactions, 69,600 users, 9,927 rated anime, and a 0.9172% matrix-density measurement.

Continued on next page

Table 2 continued

Step	Evidence examined	Decision logic and action	Output / handoff
6. Translate EDA into modelling constraints	Rating distribution, title activity, and user-history distribution.	Read the mean 7.81/mode 8 and 82.55% of ratings at least 7 as positive-score skew; read the median/P90/P99 user histories of 45/230/640 as unequal evidence; and read popularity concentration as a risk of uneven item support. These are reasons to report coverage and limitations, not reasons to delete active users or infer genre preferences.	Three EDA figures plus a written handoff specifying the original scale, missing-as-unknown rule, and coverage reporting.

The most consequential decision in this stage was the treatment of -1 . If it had been kept as a numerical target, the model could confuse a user who simply did not score a watched title with a user who strongly disliked it. Conversely, removing it does not claim that the user disliked the title; it declares that this project uses explicit ratings only. The duplicate rule followed the same principle: the average preserves the available explicit information while ensuring one target per matrix cell. No missing or unparseable required explicit ratings and no invalid 1–10 ratings were removed, so the change log explains exactly why the raw total fell from 7,813,737 to 6,337,234.

The next step was to ask what the cleaned matrix implied for the rest of the project. The rating distribution established that explicit scores are concentrated at the high end; the user-activity distribution established that histories are long-tailed; and the density calculation established that almost all user–anime pairs are unobserved. From these observations, the handoff to modelling specified three constraints: preserve the original 1–10 scale, report the coverage effects of any sample or minimum-interaction rule, and do not turn missing entries into zeros. The cleaned CSV files, validation assertions, change log, and EDA figures are the concrete handoff artifacts from this approach.

1.2 Modelling and evaluation approach: Trường

The modelling work converted the data handoff into a reproducible recommendation experiment. Trường first kept the data boundary visible: `ratings_clean` is the audited source from the first notebook, while `ratings_model` is a resource-aware subset created only for SVD. The method was selected only after considering the task and EDA together. The task requires estimating a user’s unseen-item preferences from explicit ratings, and the sparse matrix contains shared user–anime patterns that collaborative filtering can exploit. Biased SVD was therefore a direct, interpretable baseline; its latent factors are learned preference patterns, not manually named genres. Table 3 records the detailed sequence.

Table 3: Detailed decision record for the SVD, evaluation, and serving work led by Truong.

Step	Evidence examined	Decision logic and action	Output / handoff
1. Recheck the modelling input	The cleaned schemas, scale, sentinel removal, duplicate-key condition, and metadata role.	Treat Notebook 01 as the only source of model input and revalidate it on load. This prevents a local manual change or an incorrect raw file from silently changing the experiment. Keep metadata out of SVD fitting and reserve it for readable recommendation output.	Validated ratings_clean and anime_clean inputs.
2. Choose model scope and method	The 6.3-million-row cleaned dataset, sparse EDA results, and personal-computer resource limits.	Use deterministic sampling of 1,000,000 ratings with seed 42, followed by iterative user/item k-core filtering at five interactions. Choose biased SVD because it estimates explicit ratings from observed collaborative patterns without treating missing cells as zero.	ratings_model: 952,497 interactions, 41,370 users, and 5,985 anime; an explicit scope limitation.
3. Protect the holdout experiment	The risk that test ratings could influence fitted parameters or configuration decisions.	Split the filtered data once into 80% training and 20% outer test data before fitting. Use the fixed 50-factor, 20-epoch baseline with learning rate 0.005, regularisation 0.02, and seed 42. Keep lightweight tuning disabled, so the test set is neither a tuning set nor a source of optimistic model selection.	761,997 training ratings, 190,500 held-out ratings, and a reproducible SVD configuration.
4. Measure two different questions	Outer-holdout predictions containing true and estimated ratings for each observed test item.	Compute RMSE and MAE for rating-estimation error. Separately rank each user’s held-out predictions; define relevance as a true rating at least 8; require at least ten test items and one relevant item; then macro-average Precision@10 and Recall@10. This prevents a rating-error score from being mistaken for a Top-N retrieval score.	RMSE 1.2632, MAE 0.9625, Precision@10 0.6386, Recall@10 0.8255 for 4,802 eligible users.

Continued on next page

Table 3 continued

Step	Evidence examined	Decision logic and action	Output / handoff
5. Bound metric interpretation	The candidate set and excluded users in the ranking calculation.	Label ranking evaluation as held-out observed-item ranking, not full-catalog ranking. Users without enough test items or without a relevant held-out item are excluded under the stated rule rather than assigned artificial zero recall. Record that the random split is not temporal forecasting.	Metric charts, eligibility counts, and limitations included with results.
6. Separate evaluation from serving	The need for a final Top-10 list after the test metrics have been fixed.	Refit the same configuration on all <code>ratings_model</code> interactions only after evaluation. For each selected demonstration user, exclude every already rated model-scope anime, predict the remaining candidates, use deterministic tie-breaking, then join metadata after ranking.	<code>top_10_recommendations.csv</code> and seven reproducible demonstration profiles.

The resource-aware filtering was a modelling choice, not a claim that sparse users or anime are invalid. Sampling made the run feasible, while the iterative k-core rule ensured that every retained user and anime had minimum local evidence after the mutual removals stabilised. Its cost is a narrower evaluation population, which is why every result is tied to the 952,497-interaction model dataset rather than the full cleaned catalogue. The fixed configuration was intentionally reported as a baseline: it is repeatable and suitable for the assignment requirements, but it is not presented as an exhaustive hyperparameter search.

The evaluation reasoning then separated an honest test from a usable serving model. An outer 80/20 split was created before SVD fitting, and the held-out ratings were not used to fit parameters or choose the fixed baseline configuration. RMSE and MAE answer how close estimated ratings are to held-out ratings, whereas Precision@10 and Recall@10 answer whether relevant held-out observations are retrieved near the top under a documented observed-item protocol. Once those metrics were fixed, a separate model was refitted on all `ratings_model` interactions to score only anime that a demonstration user had not rated. This sequence prevents the displayed Top-10 lists from being confused with test-set evidence, future verified ratings, or objective measures of anime quality.

1.3 Shared integration and review

The two workflows meet at explicit review checkpoints rather than only at the final writing stage. First, Truong accepts the cleaned files only after the scale, sentinel, duplicate, and metadata assertions from Notebook 01 pass. Second, Quóc reviews the post-sampling and post-filtering counts so the report never calls `ratings_model` the entire dataset. Third, both members compare every reported number with the notebook-generated artifacts: cleaning counts and EDA summaries come from Notebook 01; configuration, split sizes, eligible-user counts, metrics, and Top-10 exports come from Notebook 02. This makes the notebook boundary a reproducibility control, not simply a division of labour.

For report integration, the group used a final claim-by-claim check. Public `anime_average_rating` remains display metadata and is never described as an SVD label or a personalised prediction. Figures are accompanied by an interpretation and a next-decision implication; metrics state their direction, data scope, and ranking protocol; and the conclusion does not generalise beyond the sampled, k-core-filtered, random-holdout experiment. Both members also verified that the contribution shares total 100%, references support the method and dataset claims, and the report/presentation use the same values. This review step is why the report presents decisions together with their reasons and limitations, rather than only listing executed code steps.

2 Introduction and problem statement

Anime streaming services need to help viewers navigate a broad catalogue without requiring them to inspect every title. The task in this project is to use a user’s previous *explicit* 1–10 ratings to estimate preferences for anime that the user has not rated, then rank those candidates to produce a Top-10 list. Missing interactions are not treated as zero ratings: they are unknown preferences.

The project has three measurable objectives. First, it prepares a defensible explicit-rating dataset while preserving a clear audit trail. Second, it evaluates rating estimates on a held-out test split using RMSE and MAE, where lower values are better. Third, it measures whether highly rated held-out anime are retrieved near the top of a ranked list using Precision@10 and Recall@10, where higher values are better. The recommendation output is intended as a personalised ranking signal, not as an assertion of objective anime quality.

3 Dataset and data preparation

3.1 Source and structure

The project uses the *Anime Recommendations Database* published on Kaggle by CooperUnion (CooperUnion, 2016). It contains two source tables. `rating.csv` has 7,813,737 rows with `user_id`, `anime_id`, and `rating`; one row represents a score assigned by one user to one anime. `anime.csv` has 12,294 rows with `title`, `genre`, `type`, `episode count`, `public average rating`, and `membership count`. Metadata is used only to make exported recommendations interpretable. In particular, `anime_average_rating` is not a training target and is not a predicted user rating.

3.2 Cleaning and validation

Table 4 records the cleaning audit. A value of -1 means that an anime was watched but not explicitly scored. It is therefore neither a dislike nor a valid numeric label and was excluded from the collaborative-filtering input. There were no missing or unparseable required rating fields. Seven duplicated user–anime keys were consolidated by their mean valid rating, producing one interaction per pair without arbitrarily retaining a row; this may yield valid fractional values such as 6.5 or 8.5. Ratings remained in the inclusive 1–10 range after validation.

Table 4: Data preparation audit for the explicit-rating matrix.

Measure	Count	Interpretation
Raw interactions	7,813,737	Source user–anime rows
Watched-but-unrated (−1) removed	1,476,496 (18.90%)	Not an explicit preference label
Invalid/missing rating rows removed	0	Required fields were complete and parseable
Duplicate pairs merged	7	Mean rating retained for one user–anime signal
Clean explicit interactions	6,337,234	Input baseline for later modeling
Unique users / rated anime	69,600 / 9,927	Coverage of the cleaned interaction matrix
Metadata anime	12,294	Display catalogue; not SVD labels
Matrix density	0.9172%	interactions/(users×rated anime)

The prepared outputs contain `ratings_clean.csv` with only `user_id`, `anime_id`, and `rating`, and `anime_clean.csv` with display metadata. Missing display fields were preserved instead of imputed; one rated anime lacks metadata, so a serving implementation needs a title/genre fallback. Notebook 01 does not sample, k-core filter, split, scale, or train a model. Keeping those choices in Notebook 02 prevents model-specific scope changes from being presented as data cleaning.

3.3 Exploratory data analysis

The EDA characterises the cleaned explicit-rating matrix; it does not evaluate the model. Figure 1 shows a positive rating distribution: the mean is 7.81, the mode is 8, and 82.55% of explicit ratings are at least 7. A high predicted score must consequently be interpreted against a scale where users tend to rate positively, not as evidence that a title is universally good.

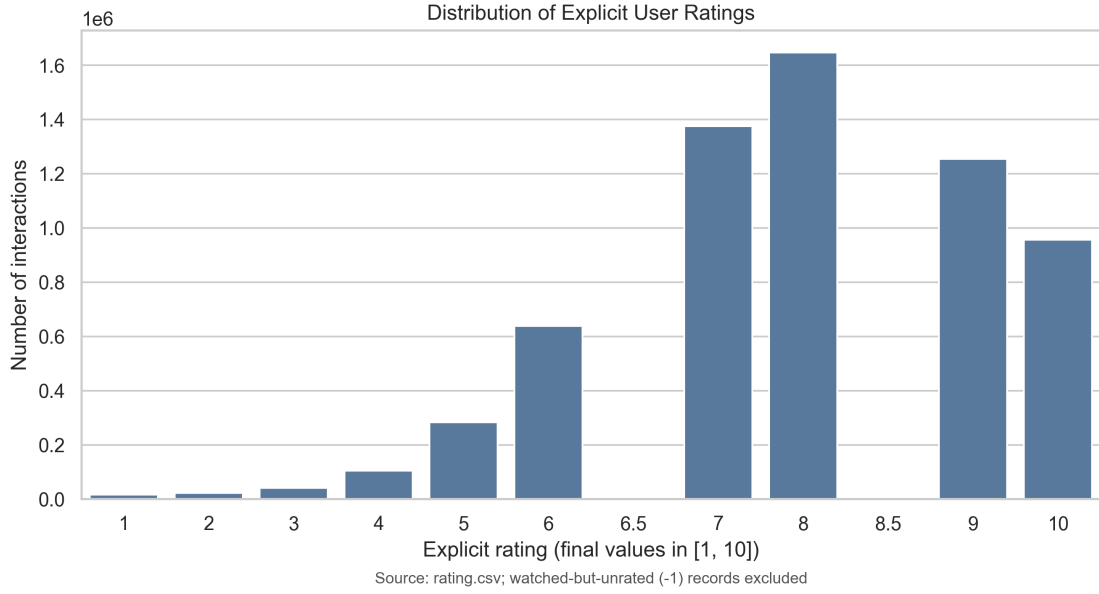


Figure 1: Distribution of explicit user ratings after excluding watched-but-unrated (-1) interactions.

Figure 2 shows the long-tail distribution of ratings per user. The median user has rated 45 anime, whereas the 90th and 99th percentiles are 230 and 640, respectively. This uneven history means that the available collaborative signal differs materially between users. It motivates reporting model coverage and any minimum-interaction rule separately, but it does not justify deleting highly active users as numerical outliers.

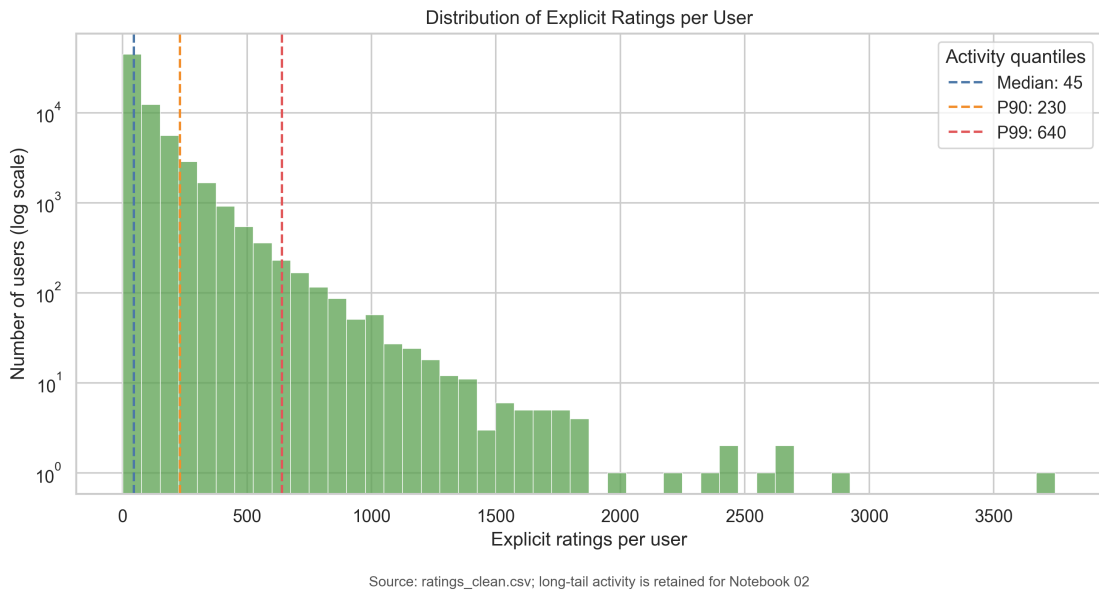


Figure 2: Long-tail distribution of explicit-rating counts per user (logarithmic frequency axis).

The 0.9172% matrix density confirms that most possible user–anime pairs are unobserved. Collaborative filtering is appropriate because it can use shared patterns among observed ratings without converting unknown pairs into zero-valued preferences. Sparsity and the long tail also signal cold-start and popularity-bias risks that qualify later results.

4 Methodology

4.1 Collaborative filtering with SVD

The system uses the biased SVD implementation in the Surprise library. This form estimates a user’s rating for anime i as

$$\hat{r}_{ui} = \mu + b_u + b_i + q_i^T p_u, \quad (1)$$

where μ is the global mean rating, b_u and b_i are user and anime biases, and p_u and q_i are learned latent-factor vectors. The model learns these terms from training ratings by minimising regularised squared error with stochastic gradient descent (Hug, n.d.; Koren et al., 2009). Latent factors are inferred preference patterns, not manually assigned genre labels.

The full cleaned dataset was retained as the auditable data baseline. For a resource-aware model run, Notebook 02 used a reproducible random sample of 1,000,000 ratings (seed 42), then iteratively removed users and anime with fewer than five interactions. The final model data comprised 952,497 interactions, 41,370 users, and 5,985 anime. This filter improves the availability of training signal but narrows coverage, so it must not be described as the whole cleaned dataset.

The fixed baseline used 50 latent factors, 20 epochs, a learning rate of 0.005, regularisation of 0.02, and random state 42. Lightweight tuning was disabled; the configuration is a reproducible baseline, not a claim of exhaustive hyperparameter optimisation. The model was trained only on the 80% outer training split (761,997 ratings); 190,500 ratings were held out for final scoring. This ordering prevents test ratings from affecting fitted parameters or configuration selection.

4.2 Evaluation protocol

RMSE and MAE compare predicted ratings with the outer holdout’s true 1–10 scores. For ranking, predictions were grouped by user and sorted by predicted rating. An item was relevant when its true held-out rating was at least 8. Precision@10 is the fraction of the ten highest-predicted held-out items that are relevant; Recall@10 is the fraction of each eligible user’s relevant held-out items recovered in that top ten. Metrics are macro-averaged over 4,802 eligible users who have at least ten test items and at least one relevant test item.

This is *held-out observed-item ranking*, not a ranking experiment over every unseen catalogue item. Of 37,748 test users, 32,911 had fewer than ten test items and 35 had no relevant test item, so they were excluded under the documented protocol rather than given artificial zero recall. For serving, a separate SVD model was refitted on all 952,497 model-data interactions and ranked only anime not already rated by the selected user.

4.3 Serving model and Top-N recommendation protocol

The evaluation model is deliberately not used to generate the final displayed recommendations. After the outer-holdout metrics were fixed, a new SVD model with the same configuration was refitted on all 952,497 interactions in `ratings_model`. For a user u , the serving candidate set contains every anime represented in that model data that u has not already rated. The system predicts $\hat{r}_{ui}$ for each candidate, sorts by predicted rating in descending order (breaking ties by anime ID), and returns the first $N = 10$ unseen titles. Thus, the output answers “which model-scope unseen anime receive the highest estimated preference?”; it does not confirm that the user will watch or rate any title.

Seven users were selected deterministically to make the output inspection reproducible across distinct interaction histories: limited history, developing history, typical history, very active, highly positive, more critical, and broad genre exposure. The user cases are examples of serving behaviour only. They are not additional test folds and are not used to calculate or validate the four evaluation metrics.

5 Implementation

The implementation is divided into two reproducible notebooks. Notebook 01 loads the raw CSV files, validates schema and missingness, audits sentinel values and duplicate keys, writes the two cleaned CSV files, and saves the EDA figures. Assertions verify the 1–10 scale, the absence of -1 , unique user–anime keys, and unique metadata IDs. This notebook intentionally stops before sampling, filtering, splitting, or training.

Notebook 02 reads only those cleaned outputs. It applies the fixed-seed sampling and k-core audit, creates the outer train/test split, fits Surprise SVD, calculates rating and ranking metrics, and saves evaluation charts. It then refits a serving model on the final model data, excludes previously rated anime from each candidate set, joins metadata with fallbacks, and exports ten recommendations per demonstration user. The two-stage design keeps evaluation honest while permitting the recommendation model to use all model-scope interactions.

6 Results and evaluation

Table 5 reports the outer-holdout results. RMSE 1.2632 and MAE 0.9625 quantify rating-prediction error on the 1–10 scale; lower is better. Precision@10 of 0.6386 and Recall@10 of 0.8255 quantify performance only under the stated relevant-rating threshold, eligible-user rule, and observed-item candidate set. They should not be interpreted as full-catalog Top-10 performance.

Table 5: SVD outer-holdout evaluation on 952,497 sampled and k-core-filtered interactions (seed 42).

Metric	Value	Direction
RMSE	1.2632	Lower is better
MAE	0.9625	Lower is better
Precision@10	0.6386	Higher is better
Recall@10	0.8255	Higher is better

Figures 3 and 4 make the two evaluation views visible. The error values show the residual scale of individual rating estimates; the ranking measures show retrieval of explicit high ratings within eligible users’ held-out observations. Neither visual demonstrates a causal effect or a guarantee that a newly recommended anime will be liked.

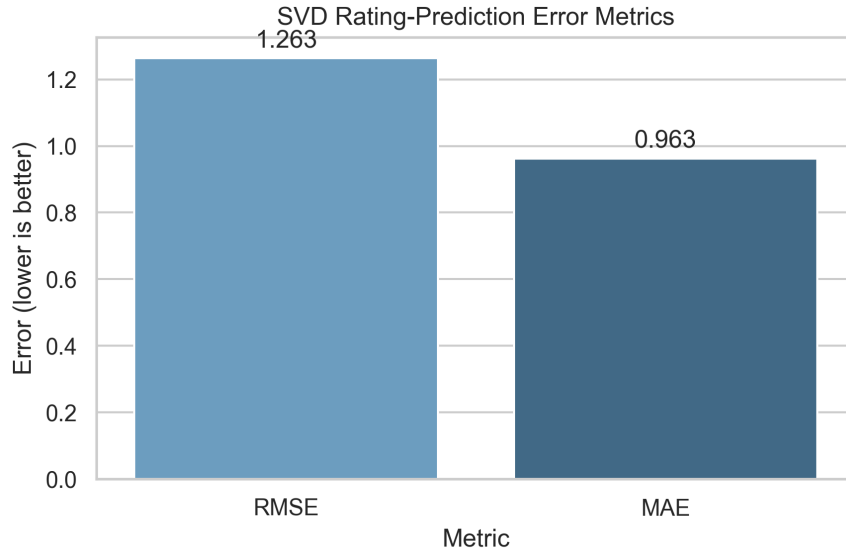


Figure 3: SVD rating-prediction error on the outer holdout (lower is better).

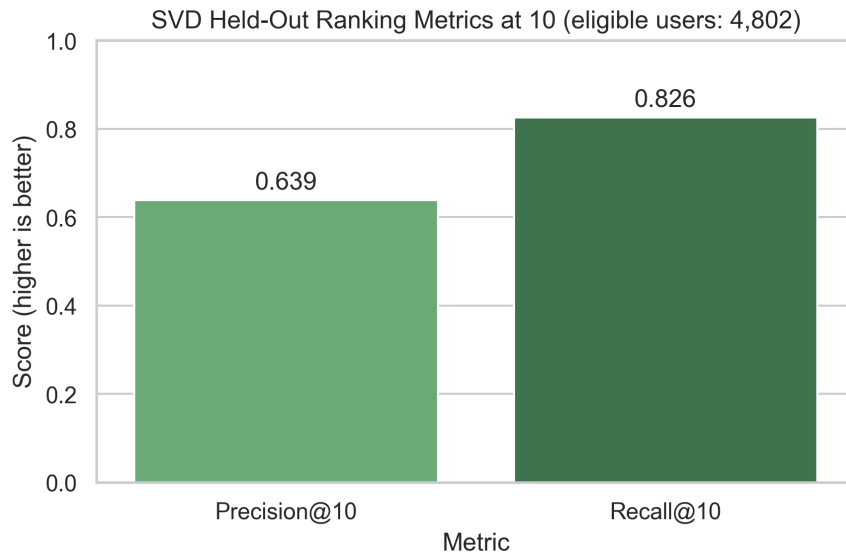


Figure 4: Held-out observed-item ranking metrics at 10 for 4,802 eligible users (higher is better).

6.1 Case-based Top-N recommendation outputs

The serving export contains ten unseen anime for each of the seven demonstration users, with title, genre, type, public average rating, and predicted rating. Table 6 describes the interaction histories used to select the cases. The *mean observed rating* and *observed genres* are profile descriptors calculated from `ratings_model`; they are not model features added to SVD.

Table 6: Deterministic demonstration profiles for the Top-10 serving outputs.

Scenario	User ID	Ratings	Mean rating	Genres
Limited history	10004	5	8.00	10
Developing history	10060	10	7.70	18
Typical history	25321	33	7.79	30
Very active	42635	513	6.38	43
Highly positive	45649	121	10.00	30
More critical	24323	45	1.60	31
Broad genre exposure	65836	255	6.26	43

The detailed predictions in Appendix 7 illustrate why a single aggregate metric cannot describe every user experience. User 10004, with only five retained ratings, receives a highly scored but necessarily less evidence-rich list. User 42635 has 513 ratings, giving the model substantially more interaction signal, yet its highest estimated preference is 8.8350 rather than a ceiling score. The highly positive profile (user 45649) receives ten estimates of 10.0000, while the more critical profile (user 24323) has a highest estimate of 5.4149. These are model estimates conditional on observed ratings and the fitted score range, not statements that one user’s recommendations are objectively better than another’s. The broad-genre user (65836) demonstrates that the list can span several genres, but the latent factors themselves must not be interpreted as named genre dimensions.

Every row in Appendix 7 is an unseen anime at the time of serving. Predicted ratings are rounded to four decimal places for presentation; the full-precision values, metadata, and all ranks are retained in `outputs/top_10_recommendations.csv`. `Public_anime_average_rating` is deliberately excluded from that appendix so it cannot be confused with the personalised SVD prediction.

7 Complete Top-10 serving outputs

This appendix reports the complete $N = 10$ unseen-anime lists exported by the serving model for all seven deterministic demonstration profiles. The lists are descriptive outputs from the refitted serving model, not holdout observations. A predicted rating of 10.0000 is an estimate at the upper bound of the configured 1–10 scale; it is not a verified future rating.

Table 7: Personalised Top-10 unseen-anime predictions for each demonstration profile.

Rank	Anime title	Predicted rating
Limited history — user 10004		
1	Ginga Eiyuu Densetsu	9.8077
2	Gintama	9.6646
3	Aria The Origination	9.6114
4	Gintama’: Enchousen	9.6038
5	Gintama Movie: Kanketsu-hen – Yorozuya yo Eien Nare	9.5993
6	Shingeki no Kyojin	9.5738
7	Katanagatari	9.5568
8	Gintama°	9.5395
9	Rainbow: Nisha Rokubou no Shichinin	9.5193

Table 7 continued

Rank	Anime title	Predicted rating
10	Mushishi Zoku Shou 2nd Season	9.4905
Developing history — user 10060		
1	Ginga Eiyuu Densetsu	10.0000
2	Kimi no Na wa.	9.6462
3	Tengen Toppa Gurren Lagann	9.6082
4	Gintama°	9.6004
5	Gintama Movie: Kanketsu-hen – Yorozuya yo Eien Nare	9.5549
6	Gintama'	9.4975
7	Mushishi	9.3837
8	Gintama': Enchousen	9.3663
9	Neon Genesis Evangelion: The End of Evangelion	9.3553
10	Fullmetal Alchemist: Brotherhood	9.3263
Typical history — user 25321		
1	Ginga Eiyuu Densetsu	9.7105
2	Neon Genesis Evangelion: The End of Evangelion	9.6636
3	Aria The Origination	9.4069
4	Gintama': Enchousen	9.2951
5	Hunter x Hunter (2011)	9.2927
6	Code Geass: Hangyaku no Lelouch R2	9.1926
7	Gintama°	9.1624
8	Kimi no Na wa.	9.1436
9	JoJo no Kimyou na Bouken (TV)	9.1095
10	Code Geass: Hangyaku no Lelouch	9.0842
Very active — user 42635		
1	Ginga Eiyuu Densetsu	8.8350
2	Aria The Origination	8.7820
3	Serial Experiments Lain	8.7504
4	Gintama': Enchousen	8.5096
5	Gintama°	8.4872
6	Tengen Toppa Gurren Lagann	8.4477
7	Azumanga Daioh	8.4399
8	Monogatari Series: Second Season	8.4074
9	Clannad: After Story	8.3053
10	Seirei no Moribito	8.3019
Highly positive — user 45649		
1	Cowboy Bebop	10.0000
2	Uchuu Kaizoku Captain Harlock	10.0000
3	Carnival Phantasm	10.0000
4	Top wo Nerae 2! Diebuster	10.0000
5	Ore no Imouto ga Konnani Kawaii Wake ga Nai Specials	10.0000
6	Coquelicot-zaka kara	10.0000
7	Aa! Megami-sama! (TV) Specials	10.0000
8	Bakuman. 2nd Season	10.0000
9	Toriko	10.0000
10	Kanojo to Kanojo no Neko	10.0000
More critical — user 24323		

Table 7 continued

Rank	Anime title	Predicted rating
1	Mobile Suit Gundam Seed Destiny	5.4149
2	Sen to Chihiro no Kamikakushi	4.7674
3	Clannad: After Story	4.5838
4	Naruto: Shippuuden Movie 2 - Kizuna	4.5431
5	Mobile Suit Gundam Seed	4.4712
6	Watashi ga Motenai no wa Dou Kangaetemo Omaera ga Warui!	4.4710
7	Shinsekai yori	4.4677
8	Fairy Tail	4.3989
9	Gungrave	4.3516
10	Ginga Eiyuu Densetsu	4.3406
Broad genre exposure — user 65836		
1	Cowboy Bebop	10.0000
2	Death Note	10.0000
3	Ginga Eiyuu Densetsu	9.4676
4	Shinsekai yori	9.4242
5	Steins;Gate	9.4122
6	Mushishi	9.2666
7	Hajime no Ippo	9.1375
8	Clannad: After Story	9.0899
9	Tengen Toppa Gurren Lagann	9.0542
10	Ping Pong The Animation	9.0004

8 Conclusion and lessons learned

This project produced a reproducible anime recommender based on explicit ratings and biased SVD. The data audit preserves the meaning of watched-but-unrated interactions, removes duplicate user–anime ambiguity, and exposes the sparse, positively skewed, long-tail structure that affects recommendation quality. On the documented sampled and filtered outer holdout, the model achieved RMSE 1.2632, MAE 0.9625, Precision@10 0.6386, and Recall@10 0.8255.

As students who had previously learned recommender-system ideas mainly in theory, our first important lesson was that a model cannot compensate for an unclear definition of the data. At first, it is tempting to regard every row in a ratings file as a training example and to begin fitting SVD immediately. Working with the dataset showed why that is unsafe: the value -1 means “watched but not rated,” not a low preference, and repeated user–anime pairs do not directly form one matrix cell. We learned to ask what each value represents before applying a transformation, to validate the result with assertions, and to keep an audit trail of every removed or merged row. In other words, data cleaning is not only a technical preparation step; it is a modelling decision that determines the meaning of the signal learned by the recommender.

The second lesson was to connect the mathematical model to the actual structure of the data. In class, collaborative filtering and SVD can appear as a compact equation with latent factors. During the project, the sparse matrix, strongly positive ratings, and unequal user histories made the practical consequences visible. Missing ratings are unknown rather than zero, a user with five ratings provides much less evidence than a user with hundreds, and a high predicted score must be interpreted against a scale on which many users already give high scores. We therefore learned that latent factors are useful learned patterns rather than automatically interpretable

genres, and that a reasonable baseline needs limitations such as cold start, popularity imbalance, and incomplete coverage to be reported alongside its predictions.

The third lesson concerned experimental design and evaluation. We learned that obtaining a numerical score is not the same as demonstrating a useful recommender. The outer train/test split has to be created before fitting so that held-out ratings do not influence the model, and the final serving model should be refitted only after the evaluation result is fixed. RMSE and MAE taught us about the error of estimated ratings, while Precision@10 and Recall@10 taught us a different question: whether relevant held-out items appear near the top of a ranked list. Implementing both views made clear that metrics answer only the question defined by their protocol. In this project, ranking is measured over held-out observed items for eligible users, so it must not be described as full-catalog Top-10 performance or as a guarantee that a user will like every suggested anime.

Finally, the project taught us the value of reproducible teamwork. Fixed random seeds, saved cleaned files, documented sampling and k-core filtering, notebook-generated figures, and cross-checking the report against exported artifacts made it possible to explain where each result came from. These practices were more demanding than simply running a notebook once, but they helped us distinguish the full cleaned dataset from the smaller model dataset and avoid claims beyond the evidence. Overall, the project moved our understanding from “how to train an SVD recommender” to “how to define, build, test, interpret, and communicate a recommender responsibly.” The current model is a baseline rather than a finished product, but the lessons from data semantics, sparse-data behaviour, honest evaluation, and reproducibility provide a practical foundation for future work such as stronger model comparison, temporal validation, and cold-start solutions.

References

- CooperUnion. (2016). *Anime recommendations database* (Data set). Data set. Kaggle. Retrieved July 16, 2026, from <https://www.kaggle.com/datasets/CooperUnion/anime-recommendations-database>
- Hug, N. (n.d.). *Matrix factorization-based algorithms*. Surprise. Retrieved July 16, 2026, from https://surprise.readthedocs.io/en/stable/matrix_factorization.html
- Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix factorization techniques for recommender systems. *Computer*, 42(8), 30–37. <https://doi.org/10.1109/MC.2009.263>